CSB24084(Assignment 6)

```
import java.util.*;
import java.util.stream.Collectors;

public class StudentPerformanceAnalyzer {

    static class Student {
        private int id;
        private String name;
        private List<String> courses;
        private Map<String, Integer> scores;

        public Student(int id, String name, List<String> courses, Map<String, Integer> scores) {
            this.id = id;
            this.name = name;
            this.courses = courses;
            this.scores = scores;
        }

        public int getId() {
            return id;
        }

        public String getName() {
            return name;
        }

        public List<String> getCourses() {
            return courses;
        }

        public Map<String, Integer> getScores() {
            return scores;
        }

        public double getAverageScore() {
            if (courses.isEmpty()) return 0.0;

            int total = courses.stream()
                    .mapToInt(course -> scores.getOrDefault(course, 0))
                    .sum();

            return (double) total / courses.size();
```

```
    }
}

public static List<Student> getTopNStudents(List<Student> students, int n) {
    return students.stream()
            .sorted(Comparator.comparingDouble(Student::getAverageScore).reversed())
            .limit(n)
            .collect(Collectors.toList());
}

public static Map<String, Double> getAverageScorePerCourse(List<Student> students) {

    Set<String> allCourses = getAllUniqueCourses(students);
    Map<String, Double> averageMap = new HashMap<>();

    for (String course : allCourses) {
        double avg = students.stream()
                .mapToInt(student -> student.getScores().getOrDefault(course, 0))
                .average()
                .orElse(0.0);

        averageMap.put(course, avg);
    }

    return averageMap;
}

public static Set<String> getAllUniqueCourses(List<Student> students) {
    Set<String> uniqueCourses = new HashSet<>();
    students.forEach(student -> uniqueCourses.addAll(student.getCourses()));
    return uniqueCourses;
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);
    System.out.print("Enter number of students: ");
    int n = sc.nextInt();

    List<Student> students = new ArrayList<>();
    String[] coursePool = {"CS", "Math", "Physics"};

    Random random = new Random();
```

```
for (int i = 1; i <= n; i++) {

    List<String> courses = new ArrayList<>();
    Map<String, Integer> scores = new HashMap<>();

    for (String course : coursePool) {
        if (random.nextBoolean()) {
            courses.add(course);
            scores.put(course, 50 + random.nextInt(51));
        }
    }

    students.add(new Student(i, "Student" + i, courses, scores));
}

int m = coursePool.length;

long startSort = System.nanoTime();
List<Student> topStudents = getTopNStudents(students, 2);
long endSort = System.nanoTime();

long startAvg = System.nanoTime();
Map<String, Double> avgPerCourse = getAverageScorePerCourse(students);
long endAvg = System.nanoTime();

Set<String> uniqueCourses = getAllUniqueCourses(students);

System.out.println("\nTop 2 Students:");
for (Student s : topStudents) {
    System.out.println(s.getId() + " - " + s.getName()
            + " | Average = " + s.getAverageScore());
}

System.out.println("\nAverage Score Per Course:");
System.out.println(avgPerCourse);

System.out.println("\nAll Unique Courses:");
System.out.println(uniqueCourses);

double sortTime = (endSort - startSort) / 1_000_000.0;
double avgTime = (endAvg - startAvg) / 1_000_000.0;

System.out.println("\nExecution Time:");
System.out.println("Course Average Computation: " + avgTime + " ms");
```

```
        System.out.println("Sorting Time: " + sortTime + " ms");

        double avgOperations = n * m;
        double sortOperations = n * (Math.log(n) / Math.log(2));

        System.out.println("\nCalculated Time Complexity Based on Input:");
        System.out.println("Course Average = " + n + " * " + m +
                " = " + (int) avgOperations + " operations (O(n * m))");

        System.out.println("Sorting = n * log2(n) = " +
                (int) sortOperations + " operations (O(n log n))");
    }
}
```

Output

```
Enter number of students: 100

Top 2 Students:
16 - Student16 | Average = 98.0
90 - Student90 | Average = 98.0

Average Score Per Course:
{CS=39.06, Math=40.3, Physics=34.59}

All Unique Courses:
[CS, Math, Physics]

Execution Time:
Course Average Computation: 6.2903 ms
Sorting Time: 17.7464 ms

Calculated Time Complexity Based on Input:
Course Average = 100 * 3 = 300 operations (O(n * m))
Sorting = n * log2(n) = 664 operations (O(n log n))
```